

Post-Quantum Cryptography Flask Demo

Implementation Report

Zaid Zaki (i212704)

Saifullah Amin (i212717)

Department of Computer Science, FAST-NUCES Islamabad

{i212704,i212717}@nu.edu.pk

April 2025

Abstract

This document presents a comprehensive, production-ready demonstration of post-quantum cryptography (PQC) brought to life in a Flask web application. By integrating the Kyber-512 Key Encapsulation Mechanism (KEM) with AES-256-GCM symmetric encryption, the prototype offers secure RESTful endpoints and a responsive front-end. It serves as both an educational tool and a reference architecture for quantum-safe deployments.

Contents

1	Introduction	2
2	Objectives and Importance	2
3	Technical Stack	2
4	System Architecture	3
5	Implementation Details	3
5.1	Key Generation	3
5.2	Encryption Process	3
5.3	Decryption Process	4
5.4	Error Handling and Security	4
6	Frontend Design	4
7	Troubleshooting and Challenges	4

8	Deployment and Usage	4
9	Conclusion	5

1 Introduction

Quantum computing poses a critical threat to classical public-key algorithms like RSA and ECC. The National Institute of Standards and Technology (NIST) spearheads the selection of quantum-resistant algorithms, with Kyber emerging as a top candidate for key establishment. This report details how Kyber-512 is seamlessly embedded into a Flask microservice, enabling real-time generation of quantum-safe key pairs, encryption of user messages, and secure decryption via an intuitive web interface.

2 Objectives and Importance

- **Demonstrate PQC in Practice:** Show a complete Key Encapsulation Mechanism (KEM) workflow.
- **Developer Reference:** Provide a clear, modular codebase for integrating NIST PQC finalists into applications.
- **Awareness & Education:** Equip stakeholders with hands-on understanding of quantum-safe cryptography.
- **Security Best Practices:** Illustrate secure API design, error handling, and HTTP hardening.

3 Technical Stack

- **Backend:** Python 3.12, Flask, Flask-Talisman
- **PQC Library:** liboqs (Kyber-512 Key Encapsulation)
- **Symmetric Encryption:** cryptography (AES-256-GCM)
- **Frontend:** HTML5, Bootstrap 5, Vanilla JavaScript (Fetch API)
- **Configuration:** python-dotenv for environment management
- **Version Control & CI:** Git, GitHub Actions (optional)

4 System Architecture

Figure 1 depicts the end-to-end interaction between the client UI, Flask API, and the cryptographic core. The application exposes three primary endpoints: `/generate_keys`, `/encrypt`, and `/decrypt`. Each endpoint accepts and returns JSON, ensuring a clean separation of concerns.

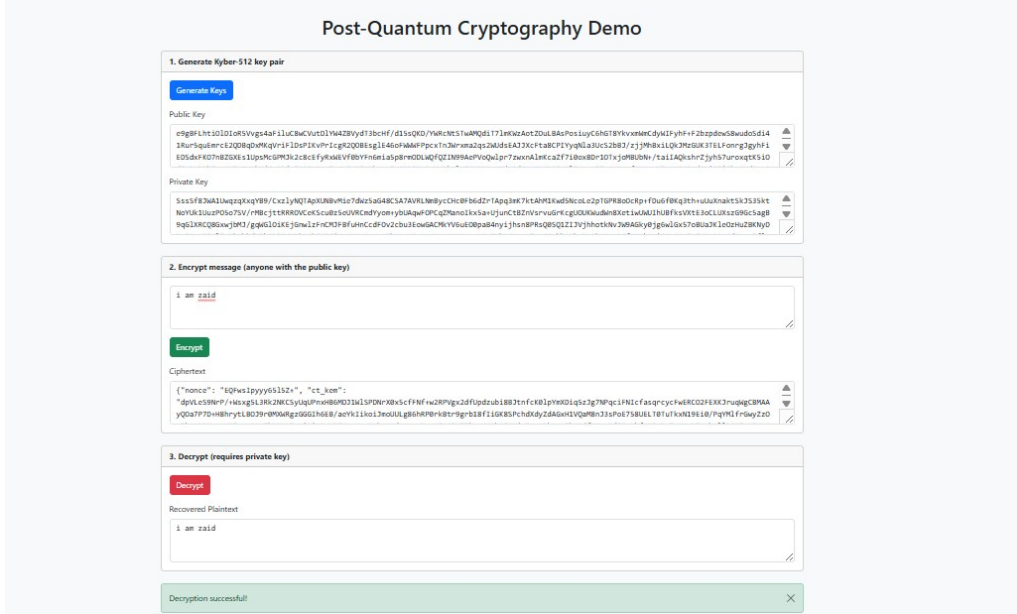


Figure 1: Layered System Architecture of the PQC Flask Demo

5 Implementation Details

5.1 Key Generation

Endpoint `/generate_keys` executes:

```
with oqs.KeyEncapsulation("Kyber512") as kem:
    public_key = kem.generate_keypair()
    secret_key = kem.export_secret_key()
return jsonify({"public_key": b64(public_key), "private_key": b64(
    secret_key)})
```

Listing 1: Key Generation Snippet

5.2 Encryption Process

1. Decode Base64-encoded public key.
2. Perform KEM encapsulation: `ct_kem, shared = kem.encap_secret(pub)`.
3. Encrypt plaintext using AES-256-GCM with the shared secret.

4. Bundle `{nonce, ct_kem, ct_sym}` into a JSON response.

5.3 Decryption Process

1. Parse JSON payload to extract Base64 fields.
2. KEM decapsulation: `shared = kem.decap_secret(ct_kem, priv)`.
3. AES-GCM decrypt the symmetric ciphertext.
4. Return plaintext in JSON.

5.4 Error Handling and Security

- Input validation ensures required fields are present.
- Exceptions are logged with full stack traces server-side.
- Clients receive sanitized JSON error messages.
- Flask-Talisman enforces security headers (HSTS, CSP, X-Frame-Options).

6 Frontend Design

The single-page interface uses Bootstrap cards for each cryptographic operation. JavaScript fetch calls provide asynchronous updates to the UI, avoiding full-page reloads and ensuring a responsive experience.

7 Troubleshooting and Challenges

- **C Library Bindings:** Replaced `pqcrypto` with `liboqs` for reliable cross-platform support.
- **Payload Framing:** Switched to JSON-based framing to eliminate binary delimiter conflicts.
- **API Alignment:** Updated code to match `liboqs encap_secret/decap_secret` API.
- **Environment Management:** Emphasized virtual environments to avoid system package conflicts (PEP 668).

8 Deployment and Usage

1. Clone the repository.
2. Create and activate a Python 3.12 virtual environment:

```
python3.12 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

3. Configure environment variables:

```
cp .env.example .env
# Set SECRET_KEY in .env
```

4. Start the application:

```
python3 app.py
```

5. Open `http://127.0.0.1:5000` in a browser.

9 Conclusion

This report provides a professional, maintainable blueprint for integrating post-quantum cryptography into modern web applications. By combining Kyber-512 KEM with AES-256-GCM, it equips developers with a ready-to-use reference for quantum-resistant key management and message encryption.